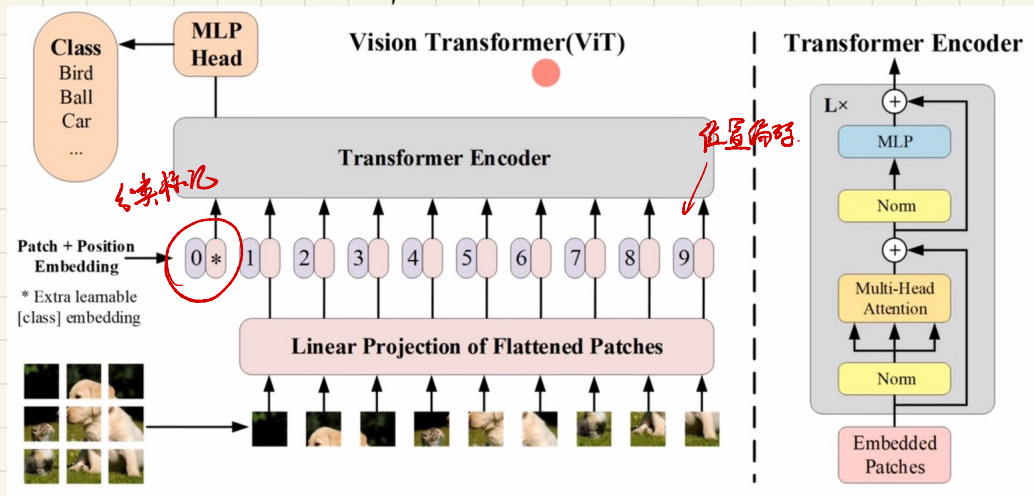
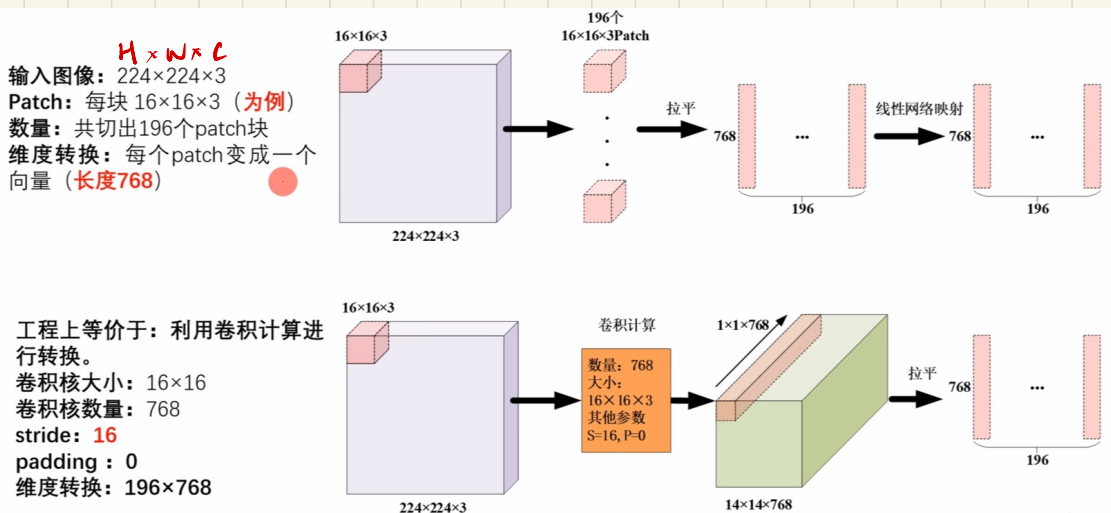


一、ViT 整体结构分析



二、ViT 模型的输入



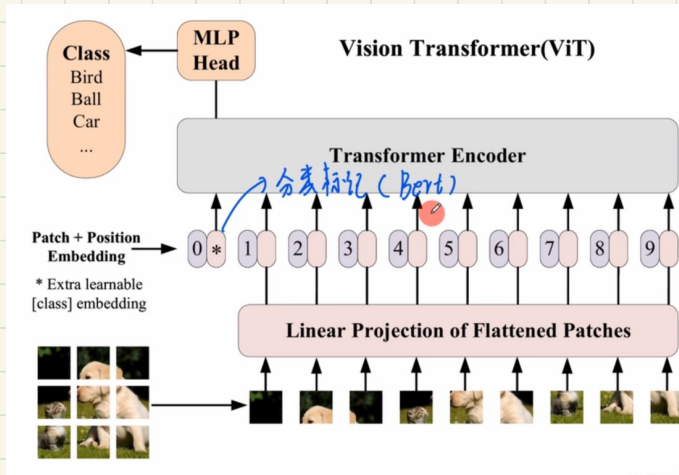
信息浓缩: 把 $16 \times 16 \times 3 = 768$ 个像素信息, 浓缩成 768 个高语义特征值 (维度相同, 但信息被提炼了)。

并行高效: GPU 可以同时计算所有核、所有位置, 速度极快。

学习能力强: 这 768 个卷积核的权重不是人工设计的, 而是在训练过程中通过反向传播自动学习出来的。模型会自己找到对当前任务 (比如分类) 最有用的那 768 种特征模式。

总结: 不是依次操作, 而是 768 个并行的特征提取器, 同时对图像的每个 Patch 进行打分, 生成一个 768 维的特征向量。

3. ViT 中的分类标记



cls token分类标记:

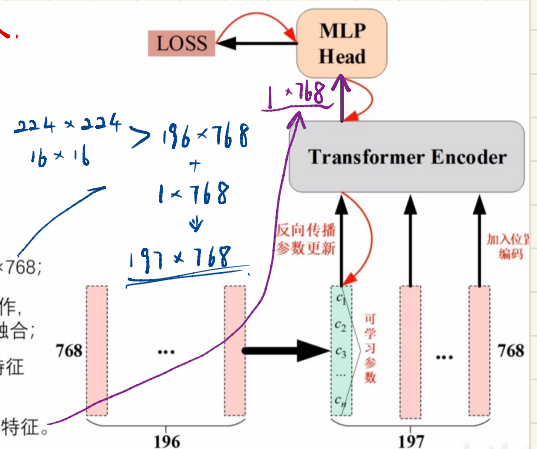
本质模型考教

本质: 一个可学习向量 c , 不来自图像内容

- 1、随机初始化为参数, 维度与模型输入一致;
- 2、每个batch复制同一cls token;
- 3、通过反向传播进行参数更新。

具体的:

- 1、初始化cls向量: 创建可学习参数, 假设维度为 1×768 ;
- 2、构造输入序列: 图像序列为和cls向量做concat操作, 获得 197×768 维度的向量。该向量和位置编码信息融合;
- 3、ViT 计算: 输入数据经过ViT特征提取后, 输出特征形状保持不变, 依旧为 197×768 维度的向量;
- 4、分类输出: 取第0个 token (cls 输出) 作为整图特征。



这 1×768 融合了后面的 196×768 信息

四. ViT 位置编码

为什么ViT必须要位置编码？

ViT把2D图像切成patch后，变成1D序列输入。此时如果不额外告诉模型“每个 patch 在图像中的位置”，自注意力只看到一堆token，很难区分空间结构。论文在方法部分明确：需要把位置编码加到patch embedding上以保留位置信息。

ViT把图像切成patch后得到个N个token+cls，然后引入一个可学习矩阵：

$$E_{pos} \in \mathbb{R}^{(N+1) \times D} \quad 197 \times 768$$

它的每一行就是一个位置的向量（第0行给cls，其余给patch）。

前向时直接相加：

$$Z_0 = \text{tokens} + E_{pos}$$

论文明确说他们使用“standard learnable 1D position embeddings”（并提到更复杂的 2D-aware 设计收益不明显）

不加位置编码

1维位置编码

2维位置编码

相对位置编码

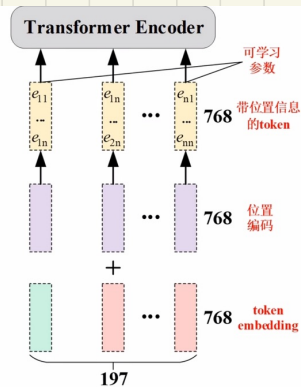
输入端加一次

每层开头都加

每层都加

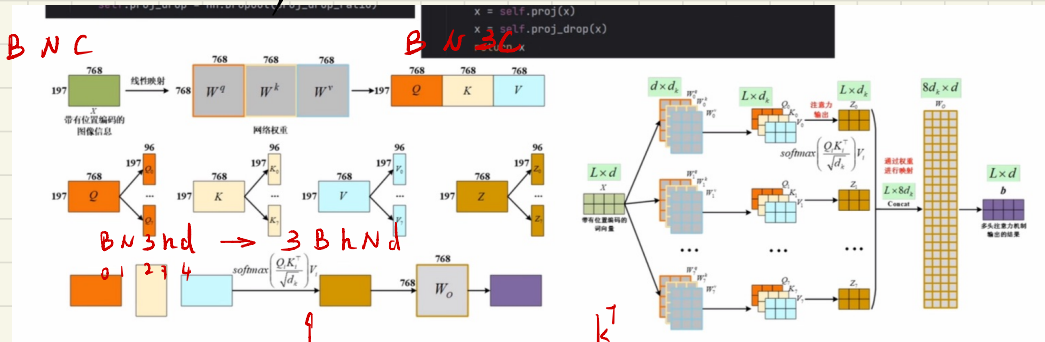
位置编码很重要

用哪种位置编码差别不大



Pos. Emb.	Default/Stem	Every Layer	Every Layer-Shared
No Pos. Emb.	0.61382	N/A	N/A
1-D Pos. Emb.	0.64206	0.63964	0.64292
2-D Pos. Emb.	0.64001	0.64046	0.64622
Rel. Pos. Emb.	0.64032	N/A	N/A

五. 多头注意力机制



$$B \times h \times d \quad @ \quad B \times h \times d \times N = B \times h \times N \times N$$

$$B \times h \times N \times N \quad @ \quad B \times h \times N \times d = B \times h \times N \times d$$

```
class Attention(nn.Module):
    def __init__(self,
                  dim,          # token embedding 维
                  num_heads=8,  # 多头数 h
                  qkv_bias=False, # qkv 线性层是否带 bias
                  qk_scale=None, # 可选，自定义缩放因子
                  attn_drop_ratio=0., # attention 权重 dropout
                  proj_drop_ratio=0.): # 输出投影 dropout
        super(Attention, self).__init__()

        self.num_heads = num_heads
        self.head_dim = dim // num_heads # 每个头的维数
        self.scale = qk_scale or head_dim ** -0.5 # 缩放

        # 一次线性层同时生成 Q,K,V (更高效)
        # 输入 [B,N,C] -> 输出 [B,N,3C]
        self.qkv = nn.Linear(dim, dim * 3, bias=qkv_bias)
        self.attn_drop = nn.Dropout(attn_drop_ratio)

        # 多头 concat 后再做一次输出投影 Wo
        self.proj = nn.Linear(dim, dim)
        self.proj_drop = nn.Dropout(proj_drop_ratio)
```

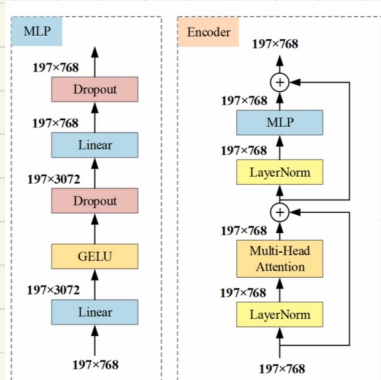
```
def forward(self, x):
    # x: [B, N, C] N = patch数 + 1 (cls), C = dim
    B, N, C = x.shape

    # 1) 生成 qkv: [B,N,3C]
    # 2) reshape: [B,N,3,h,d] (d = C/h)
    # 3) permute: [3,B,h,N,d] 方便拆出 q,k,v
    qkv = self.qkv(x).reshape(B, N, 3, self.num_heads,
                               C // self.num_heads).permute(2, 0, 3, 1, 4)
    # q,k,v: [B,h,N,d]
    q, k, v = qkv[0], qkv[1], qkv[2] # make torchscript happy

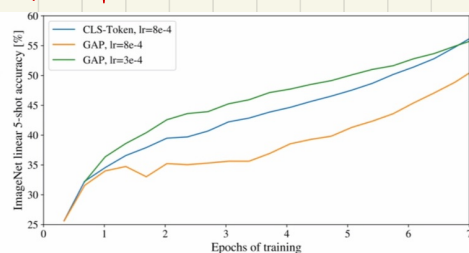
    # 加权求和得到每个 token 的新表示:
    # attn @ v: [B,h,N,d]
    # transpose: [B,N,h,d]
    # reshape: [B,N,C] (把多头拼回去)
    attn = (q @ k.transpose(-2, -1)) * self.scale # 缩放
    attn = attn.softmax(dim=-1) # 对最后一维
    attn = self.attn_drop(attn) # dropout

    # 输出投影 (对应 Wo): [B,N,C] -> [B,N,C]
    x = (attn @ v).transpose(1, 2).reshape(B, N, C)
    x = self.proj(x)
    x = self.proj_drop(x)
    return x
```

六. ViT主干结构



可接神经网络



Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

七. 论文实验整体分析

JFT: 在JFT-300M上预训练;

I21k: 在ImageNet-21k上预训练;

ViT-H/14或16: 把输入图像切成 14×14 或者 16×16 的小块, patch更小token更多, 细节保留更好, 但计算更贵;

ResNet152x4: ResNet有152层。 $\times 4$ 为通道数整体放大4倍;

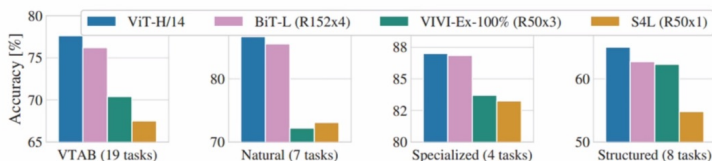
Noisy Student: 一种teacher-student半监督框架;

EfficientNet-L2: 是EfficientNet系列里一个超大规模的网络变体;

结论: 1、更大规模的预训练数据很关键; 2、patch更细通常更强, 但更耗算力; 3、ViT的算力性价比突出(多个数据集上过强CNN基线, 但预训练成本远低于当时的强基线)

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 $\pm$ 0.04	87.76 $\pm$ 0.03	85.30 $\pm$ 0.02	87.54 $\pm$ 0.02	88.4/88.5*
ImageNet Real	90.72 $\pm$ 0.05	90.54 $\pm$ 0.03	88.62 $\pm$ 0.05	90.54	90.55
CIFAR-10	99.50 $\pm$ 0.06	99.42 $\pm$ 0.03	99.15 $\pm$ 0.03	99.37 $\pm$ 0.06	—
CIFAR-100	94.55 $\pm$ 0.04	93.90 $\pm$ 0.05	93.25 $\pm$ 0.05	93.51 $\pm$ 0.08	—
Oxford-IIIT Pets	97.56 $\pm$ 0.03	97.32 $\pm$ 0.11	94.67 $\pm$ 0.15	96.62 $\pm$ 0.23	—
Oxford Flowers-102	99.68 $\pm$ 0.02	99.74 $\pm$ 0.00	99.61 $\pm$ 0.02	99.63 $\pm$ 0.03	—
VTAB (19 tasks)	77.63 $\pm$ 0.23	76.28 $\pm$ 0.46	72.72 $\pm$ 0.21	76.29 $\pm$ 1.70	—
TPUV3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k

注: 2.5k意思是1个TPUV3 core训练 2500天, 大致相当于16GB显存, 半张V100显卡的算力速度



VIVI-Ex-100%: 一种用ResNet同时在ImageNet+YouTube数据上联合训练的方法
S4L: 在ImageNet上同时用“有标签的监督学习”+“利用无标签数据的半监督/自监督约束”来训练同一个分类模型。

八. 小数据阶段CNN占优, 大数据阶段ViT后劲更强